FPT Education

TRƯỜNG ĐẠI HỌC FPT

Module 3 – Advanced Dart for Flutter Development

Overview

What you will learn

- Deep OOP concepts used in Flutter internals
- Generic programming for reusable components
- How async and event loops really work in Dart
- Stream architecture behind StreamBuilder
- Repository pattern to prepare for state management

Learning Roadmap

Advanced Language Features

- Abstract classes & implicit interfaces
- Mixins
- Factory constructors
- Generics

Async mastery

- Microtask queue vs Event queue
- Future chaining vs async/await
- Streams, broadcast, async*, yield, transformers

Architecture foundation

- Model / Repository separation
- Preparing for BLoC / Riverpod concepts

Abstract Classes

- Abstract classes define structure without full behavior.
- They force subclasses to implement functions.
- In Flutter, State<T> is an abstract class - Widgets override methods to define behavior.
- Example:

```
abstract class Shape {
        double area(); // no body = must be implemented
}
class Circle extends Shape {
        final double r;
        Circle(this.r);
        double area() => 3.14 * r * r;
}
```

Implicit Interfaces

- Every class in Dart automatically defines an interface. implements forces full override.
- Example:

```
class Bird {
        void fly() => print("Bird flying");
}
class Eagle implements Bird {
        void fly() => print("Eagle gliding");
}
```

Extends vs Implements

Keyword	Meaning	Use case
extends	Inherit behavior	Modify or extend existing class
implements	Only signature, no behavior	Define contract, custom behavior
with	Mix in code units	Share code reusable blocks

Flutter real examples

- StatefulWidget extends Widget
- AnimationController implements Listenable

Mixins

- Mixins = shared behavior without inheritance chain
- Example:

```
mixin Logger {
        void log(String msg) => print("[LOG] $msg");
}
class AuthService with Logger {
        void login() { log("login called"); }
}
```

- **Flutter uses mixins for animation, scrolling, lifecycle support.**

Mixin Constraints

- Limit mixin usage to specific parent type → ensures capability exists.
- Example:

```
class Pet { String name; Pet(this.name); }
mixin CanBark on Pet {
        void bark() => print("$name says woof!");
}
class Dog extends Pet with CanBark {
        Dog():super("Buddy");
}
```

Factory Constructors

- Factory = control over instance creation, instead of always returning new object.
- Control object creation, caching, JSON parse.
- Example:

```
class User {
        final String name;
        factory User.fromJson(Map j) => User._(j['name’]);
        User._(this.name); // private
}
```

- Flutter uses factories for theme, text styles, colors, widgets.

Factory Use Cases

Scenario	Example
Return existing instance	Singleton services
Convert format	.fromJson()
Check type before creation	Database adapters
Async creation	Load asset before building object

Generics Overview

- Generics = templates for types — write once, reuse safely.

  Dart: Future<T>, Stream<T>, List<T>, Map<K, V>

- Example:

  List<String> students = ["Anna", "Ben"];

  Map<String, int> scores = {"Anna": 95};

- Why important

  Compile-time type safety

  Avoids dynamic misuse

  Used everywhere in Flutter (widgets, async, forms)

Generic Class Example

- One class → many data types safely

  Like <String> dropdowns in Flutter or <Widget> lists

- Example:

```
class Box<T> {
    T value;
    Box(this.value);
    void show() => print("Value = $value");
}
void main() => Box<int>(10).show();
```

Generic Constraints

- Force generic to be subclass of certain type
- Example:

```
abstract class Animal { String sound(); }
class Cat extends Animal { sound() => "Meow"; }
class Box<T extends Animal> {
    T pet;
    Box(this.pet);
    void play() => print(pet.sound());
}
```

- Used in animation system & provider patterns

Collection-if / for / spread

- Dart provides functional collection building
- Example:

```
var base = [1,2,3];
var list = [  ...base,        // spread
if(true) 99,    // conditional insert
for (var x in base) x * 10 // loop insert
];
print(list);
```

- Output: [1, 2, 3, 99, 10, 20, 30]

Custom Exceptions

Purpose

- Domain-specific error context (instead of raw text strings)
- Example

```
class LoginException implements Exception {
        final String msg;
         LoginException(this.msg);
}
```

- Throw meaningful exceptions in repositories → easy to debug UI logic.

Event Loop & Microtasks

- Understanding async order = smoother UI.
- Dart async engine has two queues:

Queue	Priority	Handles
Microtask queue	▲ Highest	Immediate internal ops
Event queue	Normal	Futures, UI events, timers

- Microtasks run before event queue.

ScheduleMicrotask()

- Understanding queue = smooth animations + predictable async
- Example

```
import 'dart:async’;
void main() {
        print("A");
        scheduleMicrotask(() => print("micro"));
        Future(() => print("future"));
        print("B");
}
```

Output → A, B, micro, future

Future Chaining

- Chain async steps without nesting
- Use when you don't need variables or async/await blocks.

- Example:

```
Future(() => 1)
.then((v) => v + 1)
.then((v) => print(v));
```

Output → 2

Stream Generators - Controller & broadcast

- *async* and yield

```
Stream<int> nums() async* {
        for(int i=1;i<=3;i++) yield i;
}
```

- Controller & broadcast

```
var c = StreamController.broadcast();
c.stream.listen((v)=>print("A:$v"));
c.stream.listen((v)=>print("B:$v"));
c..add(1)..add(2);
```

Output → A:1 B:1

A:2 B:2

Repository Pattern Intro - Example (Future)

- Separate data fetching from presentation/UI

  UI -> Repository -> Data (API/DB)

- Testable, Maintainable, Works with Bloc, Riverpod, Provider
- Example (Future): UI calls: await repo.getUser()

```
class Repo {
    Future<String> getUser() async {
        await Future.delayed(Duration(milliseconds:300));
        return "Anna";
    }
}
```

→ UI calls: await repo.getUser()

Repository Stream Example

- Example:

```
Stream<int> counter() async* {
    for(int i=1;i<=3;i++){
        await Future.delayed(Duration(milliseconds:300));
        yield i;
    }
}
```

- Flutter widget

```
StreamBuilder(stream: repo.counter(), builder:... )
```

- Foundation for real-time apps

Putting It Together - Performance Best Practices

- Architecture flow

  Model -> Repo -> Stream/Future -> Widgets rebuild

- Performance Best Practices
  - Avoid blocking main isolate (no heavy loops in UI)
  - Use const widgets to reduce rebuild cost
  - Use async functions for I/O
  - Use Isolates for heavy CPU tasks (later module)

Practice Task

Goal: Build data layer simulation using Dart

- Create Product model
- Create repository returning:

  Future<List<Product>>

  Stream<Product> (live add)
- Print to console

Summary

- Advanced OOP
- Generics
- Collections
- Exceptions
- Async & Streams
- Repo pattern

References

- Mastering Flutter (2025), Chapter 2,3,6
- docs.flutter.dev
- dart.dev/tools/dartpad